

Model Development: Laptops Pricing dataset

0.1 Objectives

- Use Linear Regression in one variable to fit the parameters to a model
- Use Linear Regression in multiple variables to fit the parameters to a model
- Use Polynomial Regression in single variable to fit the parameters to a model
- Create a pipeline for performing linear regression using multiple features in polynomial scaling
- Evaluate the performance of different forms of regression on basis of MSE and R^2 parameters

Importing Required Libraries

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, r2_score
%matplotlib inline
```

Importing the dataset

```
[2]: path = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.
→cloud/IBMDeveloperSkillsNetwork-DA0101EN-Coursera/
→laptop_pricing_dataset_mod2.csv"
```

```
[3]: df = pd.read_csv(path, header=0)
```

The first 5 rows of the dataframe

```
[4]: df.head(5)
```

```
[4]:   Unnamed: 0.1  Unnamed: 0  Manufacturer  Category  GPU  OS  CPU_core  \
0           0           0         Acer         4      2   1           5
1           1           1         Dell         3      1   1           3
2           2           2         Dell         3      1   1           7
3           3           3         Dell         4      2   1           5
4           4           4          HP         4      2   1           7

      Screen_Size_inch  CPU_frequency  RAM_GB  Storage_GB_SSD  Weight_pounds_
→ \
0           14.0         0.551724         8           256         3.52800
1           15.6         0.689655         4           256         4.85100
2           15.6         0.931034         8           256         4.85100
3           13.3         0.551724         8           128         2.69010
4           15.6         0.620690         8           256         4.21155

      Price  Price-binned  Screen-Full_HD  Screen-IPS_panel
0      978         Low         0           1
1      634         Low         1           0
2      946         Low         1           0
3     1244         Low         0           1
4      837         Low         1           0
```

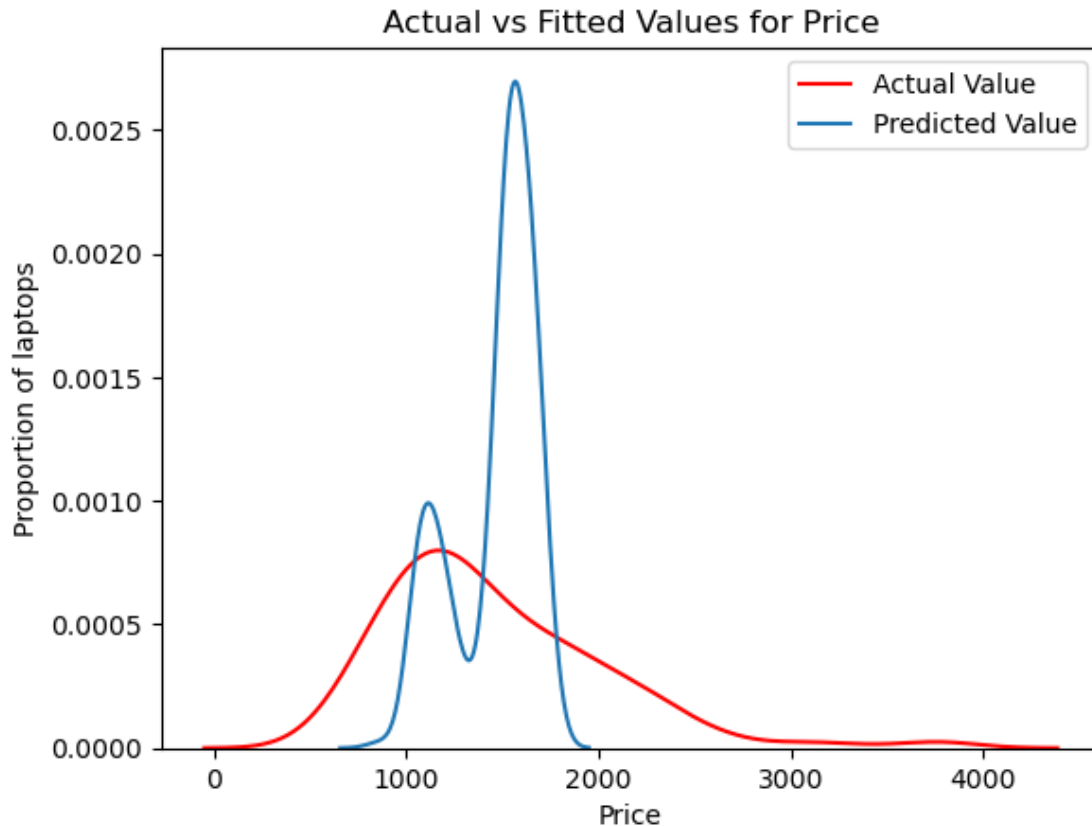
0.2 Single Linear Regression

Let's create a single feature Linear Regression model that fits the pair of "CPU_frequency" and "Price" to find the model for prediction.

```
[5]: lm = LinearRegression()
X = df[["CPU_frequency"]]
Y = df[["Price"]]
lm.fit(X,Y)
Yhat = lm.predict(X)
```

The Distribution plot for the predicted values and the actual values.

```
[6]: ax1 = sns.kdeplot(df['Price'], color='r', label='Actual value')
sns.kdeplot(Yhat, color='b', label='Predicted value', ax=ax1)
plt.title('Actual vs Fitted Values for Price')
plt.xlabel('Price')
plt.ylabel('Proportion of laptops')
plt.legend(['Actual Value', 'Predicted Value'])
plt.show()
```



Evaluation of the Mean Squared Error and R^2 score values for the model.

```
[7]: MSE = mean_squared_error(df['Price'], Yhat)
      R_2 = lm.score(X, Y)
      print('The R-square for Linear Regression is: ', R_2)
      print('The mean square error of price and predicted value is: ', MSE)
```

The R-square for Linear Regression is: 0.1344436321024326

The mean square error of price and predicted value is: 284583.4405868629

0.3 Multiple Linear Regression

Let's create a Multiple Linear Regression system for variables 'CPU_frequency', 'RAM_GB', 'Storage_GB_SSD', 'CPU_core', 'OS', 'GPU' and 'Category'.

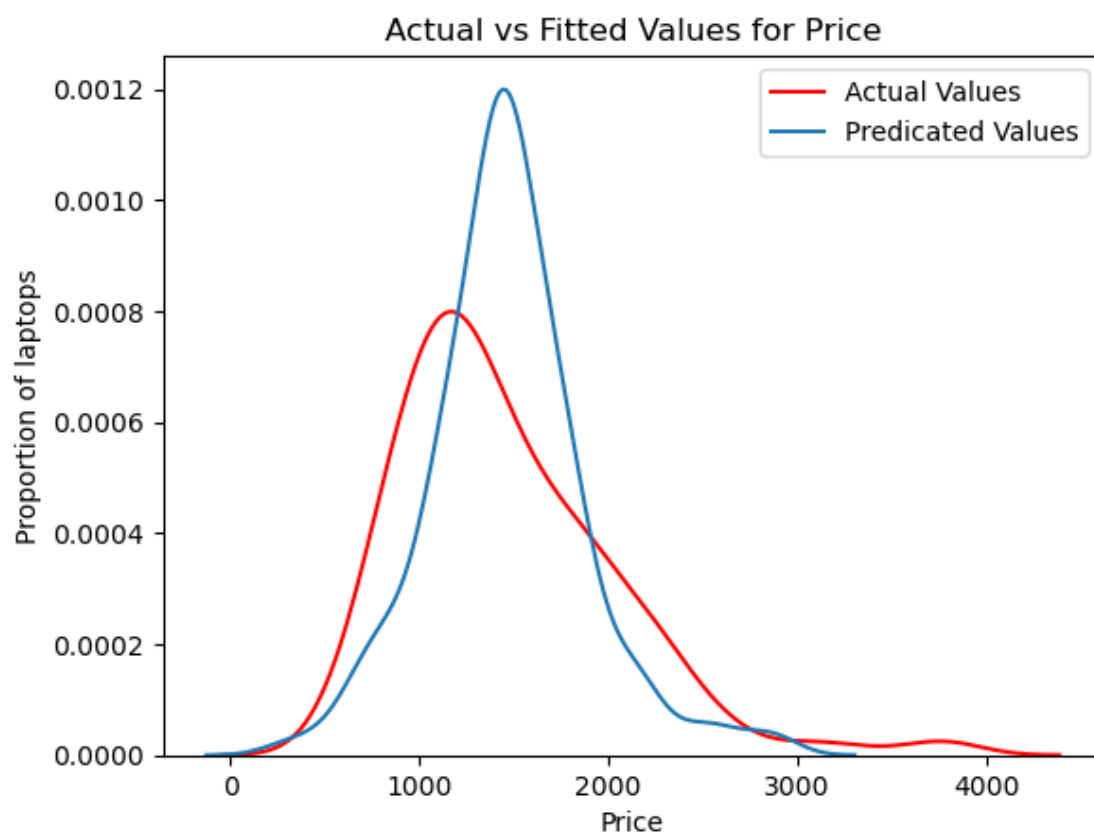
```
[8]: lm1 = LinearRegression()
      X_m = df[["CPU_frequency", 'RAM_GB', 'Storage_GB_SSD', 'CPU_core', 'OS', 'GPU', 'Category']]
      lm1.fit(X_m, Y)
```

```
Y_hat = lm.predict(X_m)
```

Let's plot the Distribution graph of the predicted values as well as the Actual values

```
[9]: ax2 = sns.kdeplot(df['Price'], color='r', label='Actual Values')
sns.kdeplot(Y_hat, color='b', label='Predicated Values', ax=ax2)
plt.title('Actual vs Fitted Values for Price')
plt.xlabel('Price')
plt.ylabel('Proportion of laptops')
plt.legend()
```

```
[9]: <matplotlib.legend.Legend at 0x772828772ad0>
```



The R^2 score and the MSE value for this fit.

```
[10]: r_2_m = lm.score(X_m, Y)
MSE_m = mean_squared_error(df['Price'], Y_hat)
print('The R-square for Multiple Linear Regression is: ', r_2_m)
print('The mean square error of price and predicted value is: ', MSE_m)
```

The R-square for Multiple Linear Regression is: 0.5082509055187374

The mean square error of price and predicted value is: 161680.57263893107

0.4 Polynomial Regression

We are using the variable “CPU_frequency” to create Polynomial features. Let’s Try this for 3 different values of polynomial degrees.

```
[11]: x = df["CPU_frequency"]
      y = df["Price"]
      f1 = np.polyfit(x, y, 1)
      p1 = np.poly1d(f1)

      f2 = np.polyfit(x, y, 3)
      p2 = np.poly1d(f2)

      f3 = np.polyfit(x, y, 5)
      p3 = np.poly1d(f3)
```

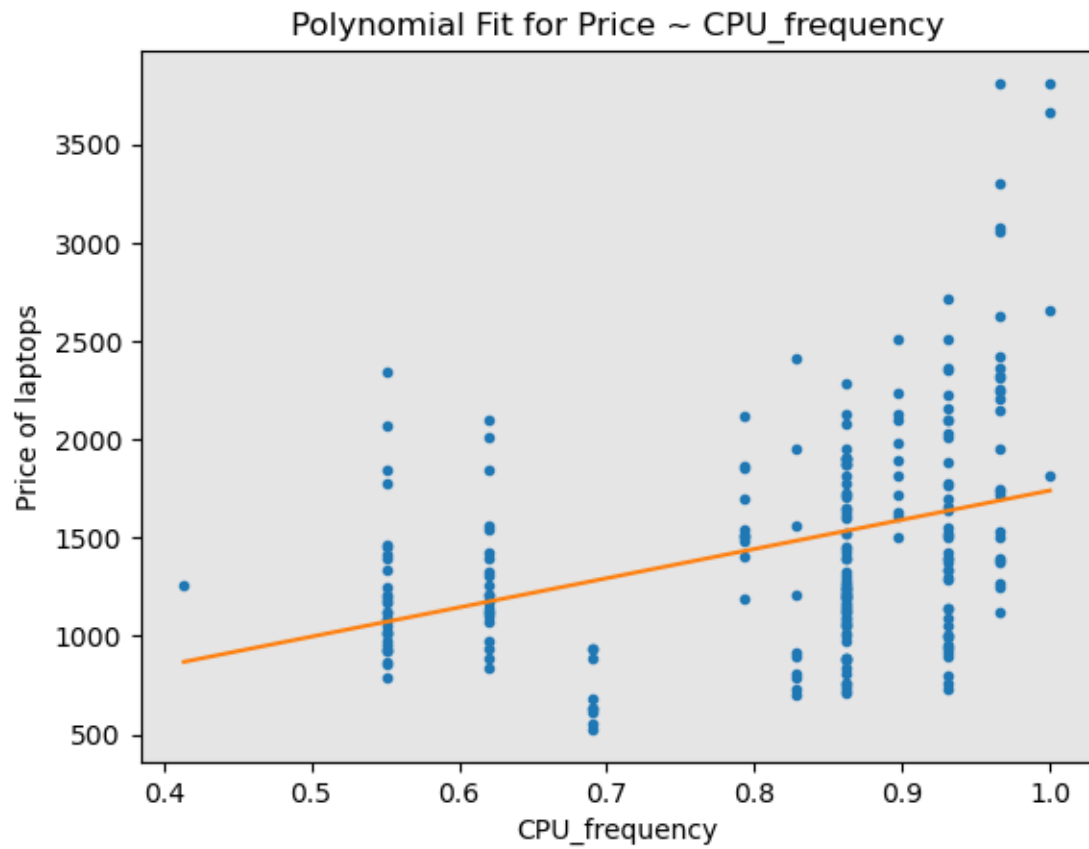
Creation of the fuction to plot the polynomial response over the actual data points.

```
[12]: def PlotPolly(model, independent_variable, dependent_variabble, Name):
      x_new = np.linspace(independent_variable.min(), independent_variable.
      ↪max(), 100)
      y_new = model(x_new)

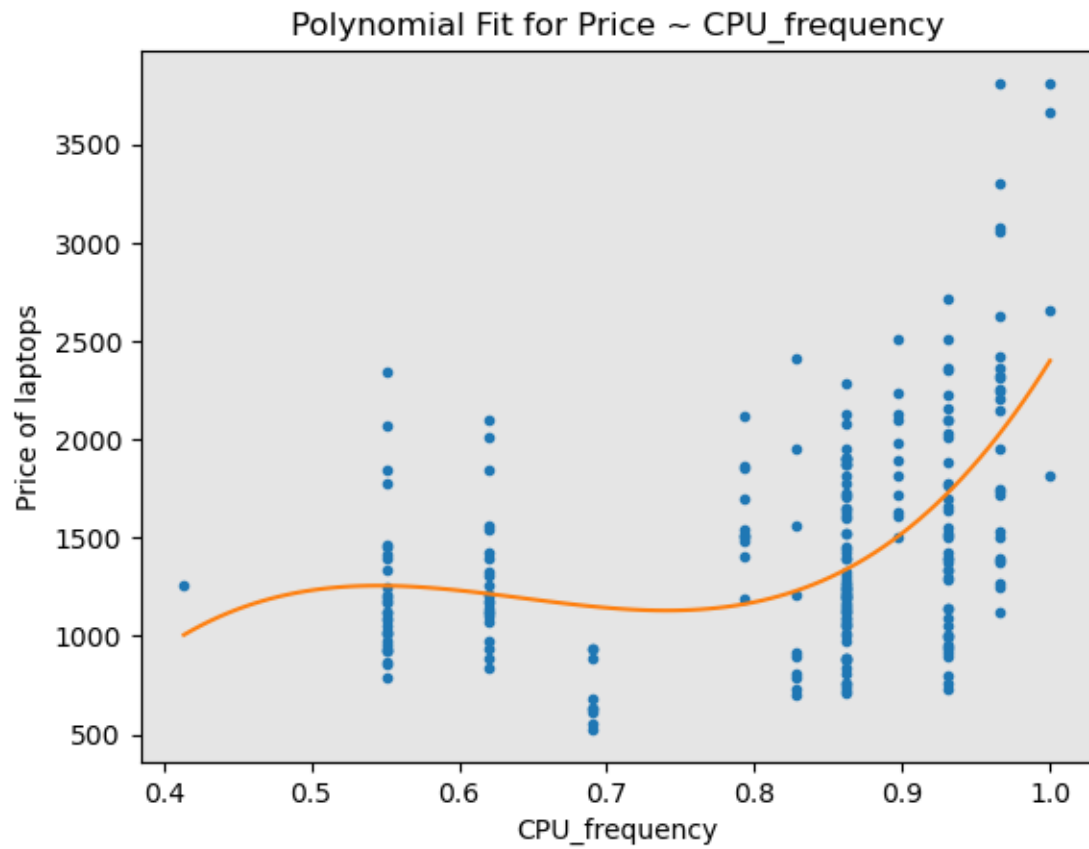
      plt.plot(independent_variable, dependent_variabble, '.', x_new,
      ↪y_new, '-')
      plt.title(f'Polynomial Fit for Price ~ {Name}')
      ax = plt.gca()
      ax.set_facecolor((0.898, 0.898, 0.898))
      fig = plt.gcf()
      plt.xlabel(Name)
      plt.ylabel('Price of laptops')
```

Calling function for the 3 models created.

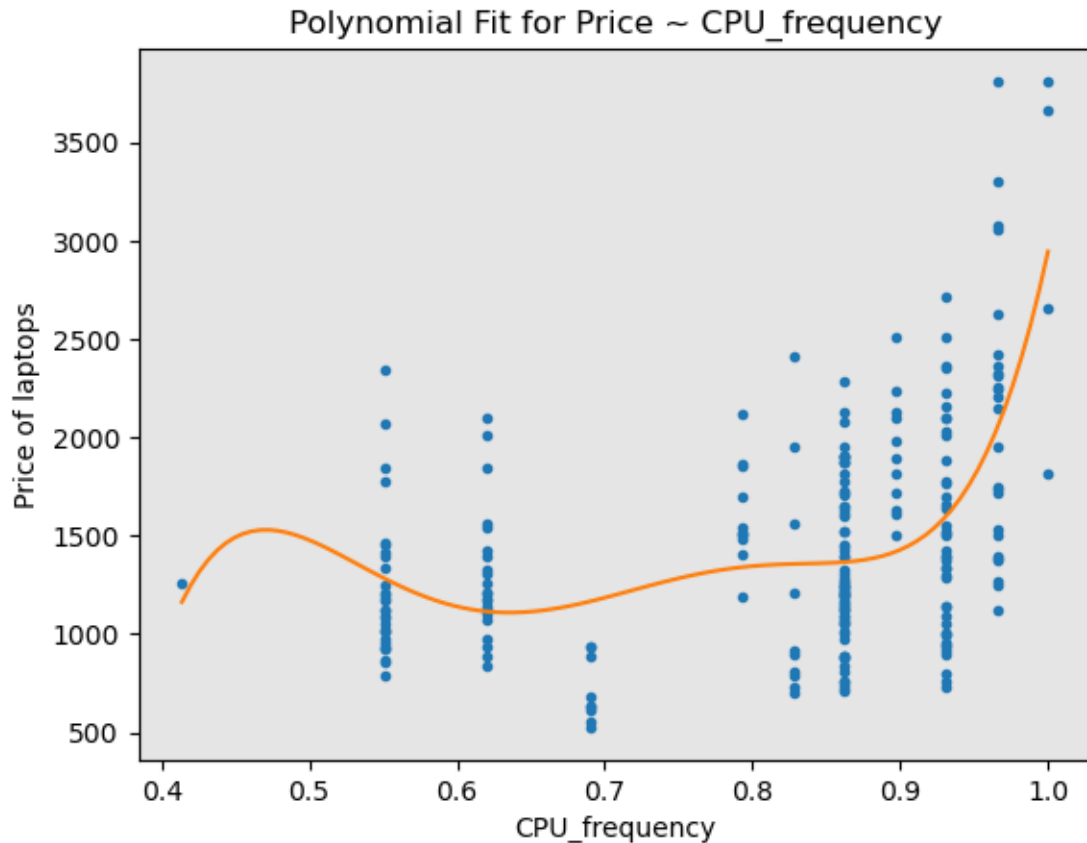
```
[13]: # function of degree 1
      PlotPolly(p1, x, y, 'CPU_frequency')
```



```
[14]: #function of degree 3
PlotPolly(p2, x, y , 'CPU_frequency')
```



```
[15]: # function of degree 5
PlotPolly(p3, x, y , 'CPU_frequency')
```



The R^2 and MSE values for these fits.

```
[18]: r_2_p1 = r2_score(y, p1(x))
MSE_p1 = mean_squared_error(y, p1(x))
print('The R-square for 1st degree polynomial is: ', r_2_p1)
print('The mean square error for 1st degree polynomial is: ', MSE_p1)
r_2_p2 = r2_score(y, p2(x))
MSE_p2 = mean_squared_error(y, p2(x))
print('The R-square for 3rd degree polynomial is: ', r_2_p2)
print('The mean square error for 3rd degree polynomial is: ', MSE_p2)
r_2_p3 = r2_score(y, p3(x))
MSE_p3 = mean_squared_error(y, p3(x))
print('The R-square for 5th degree polynomial is: ', r_2_p3)
print('The mean square error for 5th degree polynomial is: ', MSE_p3)
```

```
The R-square for 1st degree polynomial is: 0.13444363210243238
The mean square error for 1st degree polynomial is: 284583.44058686297
The R-square for 3rd degree polynomial is: 0.2669264079653123
The mean square error for 3rd degree polynomial is: 241024.8630384873
```


The R-square for 5th degree polynomial is: 0.3030822706443609

The mean square error for 5th degree polynomial is: 229137.29548054465

0.5 Pipeline

Creation of the pipeline that performs parameter scaling, Polynomial Feature generation and Linear regression.

```
[19]: Input = [('Scale', StandardScaler()), ('polynomial',
    ↳PolynomialFeatures(include_bias=False)), ('model', LinearRegression())]
pipe = Pipeline(Input)
Z = X_m.astype(float)
pipe.fit(Z, y)
ypipe = pipe.predict(Z)
```

Evaluation of the MSE and R² values for the this predicted output.

```
[24]: r_2_pm = r2_score(y, ypipe)
MSE_pm = mean_squared_error(y, ypipe)
print('The R-square for polynomial features is: ', r_2_pm)
print('The mean square error for polynomial features is: ', MSE_pm)
```

The R-square for polynomial features is: 0.6332094535859658

The mean square error for polynomial features is: 120595.86128028372

0.6 Conclusion

For multiple linear regression extended with polynomial features, we get a better R² value and mean_squared_error. Polynomial features will be a good model for this dataset

```
[ ]:
```